

Assignment 4: Registers, Stack Organization, Instruction Formats, and the Instruction Cycle

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 17, 2026

Deadline: two weeks from the date of issue — there is no automatic late penalty; if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).

Answer every question. Show all working for Numerical and Design questions — a bare final answer without the derivation earns partial credit only.

4.1 Conceptual

Q1. Registers come in two kinds: program-visible and internal. Name the three kinds of program-visible registers this course uses, and the four internal registers the control unit drives through the instruction cycle. Explain why a program can name the former but not the latter, and state which register's job is *only* to hold the address of the next instruction.

Q2. A stack can be built from registers (a register stack) or from main memory (a memory stack). Explain why real machines use memory stacks for function calls even though a register stack is faster. Your answer must name the concrete property of function calls that a small register stack cannot satisfy, and state what **SP** actually holds in each design.

4.2 Numerical

Q3. Compile the expression $Y = (A - B) \times (C + D)$ in each of the four instruction formats (3-address, 2-address, 1-address, 0-address), writing the instruction sequence for each. For the 2-address and 1-address forms, state how many instructions each needs and why any extra copy/temporary instructions are required. Give all four counts.

Q4. The stack contains `[?, 3, 8, 2]` with the top at 2. Trace the following sequence, showing the stack after every operation and the value returned by each **POP**: **PUSH 5**, **ADD**, **PUSH 1**, **POP**, **POP**. Assume **ADD** pops the top two values and pushes their sum, and the stack grows downward in memory.

4.3 Design

Q5. Using the common-bus register-transfer mechanism from the lecture, write the complete step-by-step sequence to copy the contents of register **R5** into register **R3** ($R3 \leftarrow [R5]$), mirroring the lecture's $R1 \leftarrow [R2]$ example (which used the select signals **SELA**/**SELB**/**SELD** and the bus). For

each step state which select signal encodes what, what is on the bus, and what happens on the clock edge.

Q6. On a 1-address (accumulator) machine, the instruction **ADD A** uses the accumulator as the implicit first operand and memory location **A** as the second. Write the fetch and execute phases of the instruction cycle for **ADD A**, in the same phase-by-phase table format as the lecture's **ADD R1**, A example: for Fetch give the register activity (MAR, MDR, IR, PC); for Decode state what the opcode does; for Execute give the register activity and the memory/ALU action. State which register ends up with the result.